EXPERIMENT-06

Naïve Bayes Classifier: Dataset-Based Python Programs

Name: MURE THEJASRI Reg.No: 192311298

6. Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

Problem 1: Diabetes Prediction

Aim

To implement the Gaussian Naïve Bayes algorithm on a diabetes dataset stored in a CSV file and display the confusion matrix and accuracy.

Procedure

1. Save the training examples in diabetes_data.csv with numeric attributes and the class label in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV file and split the data into 70% training and 30% testing sets.
4. Train the Gaussian Naïve Bayes model on the training set.
5. Predict the class of every sample in the test set.
6. Display the confusion matrix (rows = actual class, columns = predicted class) and the accuracy.

Dataset File / Input

File name: diabetes_data.csv

The CSV file contains a header row followed by 40 training examples. The first 10 are shown here; the complete data is in the CSV file:

Glucose	BloodPressure	BMI	Age	Outcome
119	73	27.0	39	No
145	83	34.0	40	Yes
113	63	28.6	41	No
104	85	23.0	42	No
109	69	32.8	32	Yes
93	76	26.5	39	No
114	63	29.1	33	No
120	77	23.7	35	No
123	65	31.3	36	Yes
105	66	30.3	32	No

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

df = pd.read_csv("diabetes_data.csv")
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Classes:", model.classes_)
```

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred))
print("Accuracy:", round(accuracy_score(y_test, pred) * 100, 2), "%")
```

Output

```
Classes: ['No' 'Yes']
Confusion Matrix:
[[5 1]
 [1 5]]
Accuracy: 83.33 %
```

Result

The 12 test samples give the confusion matrix [[5, 1], [1, 5]] for the classes (No, Yes). Ten samples are classified correctly, so the accuracy is 83.33%, with one false positive and one false negative.

Problem 2: Iris Species

Aim

To implement the Gaussian Naïve Bayes algorithm on a three-class iris dataset stored in a CSV file and display the confusion matrix and accuracy.

Procedure

1. Save the training examples in iris_data.csv with numeric attributes and the class label in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV file and split the data into 70% training and 30% testing sets.
4. Train the Gaussian Naïve Bayes model on the training set.
5. Predict the class of every sample in the test set.
6. Display the confusion matrix (rows = actual class, columns = predicted class) and the accuracy.

Dataset File / Input

File name: iris_data.csv

The CSV file contains a header row followed by 40 training examples. The first 10 are shown here; the complete data is in the CSV file:

SepalLength	SepalWidth	PetalLength	PetalWidth	Species
5.1	2.9	5.6	2.1	Virginica
5.7	3.3	5.3	2.0	Virginica
5.7	2.8	3.7	1.3	Versicolor
5.5	3.4	1.6	0.0	Setosa
6.9	3.2	5.3	2.0	Virginica
5.5	3.5	1.3	0.1	Setosa
4.8	3.3	1.7	0.3	Setosa
4.9	2.8	1.4	0.3	Setosa
6.4	2.8	4.5	1.3	Versicolor
5.7	2.9	4.3	1.2	Versicolor

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

df = pd.read_csv("iris_data.csv")
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
```

```
model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Classes:", model.classes_)
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred))
print("Accuracy:", round(accuracy_score(y_test, pred) * 100, 2), "%")
```

Output

```
Classes: ['Setosa' 'Versicolor' 'Virginica']
Confusion Matrix:
[[2 0 0]
 [0 3 0]
 [0 1 6]]
Accuracy: 91.67 %
```

Result

The 3 x 3 confusion matrix shows that all Setosa and Versicolor test samples are correct, and one Virginica sample is predicted as Versicolor. Eleven of 12 samples are correct, so the accuracy is 91.67%.

Problem 3: Student Result

Aim

To implement the Gaussian Naïve Bayes algorithm on a student-result dataset stored in a CSV file and display the confusion matrix and accuracy.

Procedure

1. Save the training examples in student_data.csv with numeric attributes and the class label in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV file and split the data into 70% training and 30% testing sets.
4. Train the Gaussian Naïve Bayes model on the training set.
5. Predict the class of every sample in the test set.
6. Display the confusion matrix (rows = actual class, columns = predicted class) and the accuracy.

Dataset File / Input

File name: student_data.csv

The CSV file contains a header row followed by 40 training examples. The first 10 are shown here; the complete data is in the CSV file:

StudyHours	Attendance	PreviousMarks	Result
3.4	83	60	Fail
3.8	54	66	Fail
5.2	83	60	Fail
5.0	71	54	Fail
10.2	83	47	Pass
7.7	73	58	Fail
3.4	57	68	Fail
7.4	87	72	Pass
2.9	67	44	Fail
3.3	69	58	Fail

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
df = pd.read_csv("student_data.csv")
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Classes:", model.classes_)
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred))
print("Accuracy:", round(accuracy_score(y_test, pred) * 100, 2), "%")
```

Output

```
Classes: ['Fail' 'Pass']
Confusion Matrix:
[[1 2]
 [1 8]]
Accuracy: 75.0 %
```

Result

The confusion matrix for the classes (Fail, Pass) is [[1, 2], [1, 8]]. Nine of 12 test samples are correct, so the accuracy is 75%; the model finds the Fail class harder to predict than Pass.

Problem 4: Spam Detection

Aim

To implement the Gaussian Naïve Bayes algorithm on an email dataset stored in a CSV file to separate spam from non-spam and display the confusion matrix and accuracy.

Procedure

1. Save the training examples in email_data.csv with numeric attributes and the class label in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV file and split the data into 70% training and 30% testing sets.
4. Train the Gaussian Naïve Bayes model on the training set.
5. Predict the class of every sample in the test set.
6. Display the confusion matrix (rows = actual class, columns = predicted class) and the accuracy.

Dataset File / Input

File name: email_data.csv

The CSV file contains a header row followed by 40 training examples. The first 10 are shown here; the complete data is in the CSV file:

Links	CapitalWords	Exclamations	Result
1	9	0	NotSpam
1	7	2	NotSpam
3	7	0	NotSpam
5	9	2	Spam
5	7	5	Spam
3	15	4	Spam
6	4	5	Spam
3	7	3	NotSpam
5	11	7	Spam
3	11	0	NotSpam

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

df = pd.read_csv("email_data.csv")
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Classes:", model.classes_)
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred))
print("Accuracy:", round(accuracy_score(y_test, pred) * 100, 2), "%")
```

Output

```
Classes: ['NotSpam' 'Spam']
Confusion Matrix:
[[5 1]
 [1 5]]
Accuracy: 83.33 %
```

Result

The confusion matrix for the classes (NotSpam, Spam) is [[5, 1], [1, 5]]. Ten of 12 test emails are classified correctly, so the accuracy is 83.33%.

Problem 5: Loan Approval

Aim

To implement the Gaussian Naïve Bayes algorithm on a loan dataset stored in a CSV file and display the confusion matrix and accuracy.

Procedure

1. Save the training examples in loan_data.csv with numeric attributes and the class label in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the CSV file and split the data into 70% training and 30% testing sets.
4. Train the Gaussian Naïve Bayes model on the training set.
5. Predict the class of every sample in the test set.
6. Display the confusion matrix (rows = actual class, columns = predicted class) and the accuracy.

Dataset File / Input

File name: loan_data.csv

The CSV file contains a header row followed by 40 training examples. The first 10 are shown here; the complete data is in the CSV file:

Income	CreditScore	Age	Approved
58	703	41	Yes
44	632	33	No
47	657	43	Yes
58	752	50	Yes
38	668	30	No
52	686	40	No
72	699	17	Yes
44	690	34	No
49	720	36	Yes
37	545	33	No

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

df = pd.read_csv("loan_data.csv")
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Classes:", model.classes_)
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred))
print("Accuracy:", round(accuracy_score(y_test, pred) * 100, 2), "%")
```

Output

```
Classes: ['No' 'Yes']
Confusion Matrix:
[[4 0]
 [1 7]]
Accuracy: 91.67 %
```

Result

The confusion matrix for the classes (No, Yes) is [[4, 0], [1, 7]]. Eleven of 12 test samples are correct, so the accuracy is 91.67%; one approved loan is predicted as rejected.